

more detail. Jack Diederich's PyCon 2012 talk "Stop Writing Classes" covers other ways that you might be overcomplicating your Python code.

List Comprehensions Within List Comprehensions

List comprehensions are a concise way to create complex list values. For example, to create a list of strings of digits for the numbers 0 through 100, excluding all multiples of 5, you'd typically need a `for` loop:

```
>>> spam = []
>>> for number in range(100):
...     if number % 5 != 0:
...         spam.append(str(number))
...
>>> spam
['1', '2', '3', '4', '6', '7', '8', '9', '11', '12', '13', '14', '16', '17',
--snip--
'86', '87', '88', '89', '91', '92', '93', '94', '96', '97', '98', '99']
```

Alternatively, you can create this same list in a single line of code by using the list comprehension syntax:

```
>>> spam = [str(number) for number in range(100) if number % 5 != 0]
>>> spam
['1', '2', '3', '4', '6', '7', '8', '9', '11', '12', '13', '14', '16', '17',
--snip--
'86', '87', '88', '89', '91', '92', '93', '94', '96', '97', '98', '99']
```

Python also has syntax for set comprehensions and dictionary comprehensions:

```
❶ >>> spam = {str(number) for number in range(100) if number % 5 != 0}
>>> spam
{'39', '31', '96', '76', '91', '11', '71', '24', '2', '1', '22', '14', '62',
--snip--
'4', '57', '49', '51', '9', '63', '78', '93', '6', '86', '92', '64', '37'}

❷ >>> spam = {str(number): number for number in range(100) if number % 5 != 0}
>>> spam
{'1': 1, '2': 2, '3': 3, '4': 4, '6': 6, '7': 7, '8': 8, '9': 9, '11': 11,
```